

**T.C.
İZMİR UNIVERSITY OF ECONOMICS**



FACULTY OF ENGINEERING

CE 216

“SPORTS MANAGER” PROJECT

MILESTONE 2 REPORT DOCUMENT

SİMGE GÖKALP - 20230602029

TUANA YAĞMUR - 20240602410

VİLDAN TURNALAR İNAL - 20253501001

UĞUR EMİN BAYNAL - 20220602408

LECTURER: KAYA OĞUZ

1. Introduction

This report presents the comparison between the initial system design (M1) and the first implemented version of the Sports Manager application. The purpose is to evaluate how closely the implementation follows the architectural decisions, design principles, and requirements defined in the design document, and to identify any deviations, simplifications, or improvements introduced during development.

The Sports Manager application was designed as a modular, extensible sports management framework based on object-oriented design principles. The initial design emphasized strict separation of concerns through a layered architecture, interface-based interactions, and the use of the Factory pattern to support multiple sports without modifying the core system.

The first implementation successfully realizes the core architectural vision. The system includes a functional domain model (League, Team, Player, Match), a centralized application controller (SportManager), and a working Football module that demonstrates the extensibility of the design. The Factory pattern and Sport abstraction have been correctly implemented, enabling sport-specific behavior to be encapsulated within concrete modules.

However, several simplifications and deviations were introduced during implementation to meet development constraints and ensure a working end-to-end system within the given timeframe. Features such as multi-season support, training mechanics, and full resource abstraction were deferred. Additionally, some architectural boundaries—particularly between the application and UI layers—were relaxed, with SportManager taking on both coordination and navigation responsibilities. Despite these deviations, the system maintains its extensibility and core design integrity.

2. System Architecture Overview

The Sports Manager application follows a four-layer architecture designed to separate concerns and ensure extensibility. Each layer has a well-defined responsibility, and communication between layers follows a strict one-directional downward dependency rule.

Layer 1 - User Interface Layer

This layer contains JavaFX controllers (MainMenuController, DashboardController, SquadController, MatchScreenController, FixtureController, LeagueTableController, SportSelectionController, TeamSelectionController) and their corresponding FXML views. An AppToolbarController provides shared navigation across screens. Controllers never import or reference any concrete sport class; they interact exclusively with SportManager and use abstract types such as Sport, Team, Player, Match, and StandingEntry. A dedicated SceneManager class handles all screen transitions, keeping navigation logic out of individual controllers.

Layer 2 - Core System Layer

SportManager serves as the single facade between the UI and the rest of the system. It delegates all state management to GameSession, which is implemented as a singleton

holding the current season year, week number, selected sport, managed team, and active league. The core domain model includes the abstract classes Player, Team, Match, and League, along with concrete supporting classes: MatchResult, MatchSegment, InjuryRecord, StandingEntry, Lineup, Tactic, and Coach. These classes are entirely sport-independent and provide the shared structure that all sport modules build upon.

Layer 3 - Sport Abstraction Layer

The Sport interface defines the contract that every sport module must implement. It declares methods such as getName(), createLeague(), createTeam(), createMatch(), getPositions(), getTactics(), getSegmentCount(), getSegmentLabel(), getRequiredLineupSize(), and getMaxSubstituteCount(). SportFactory is the sole point of instantiation: it receives a sport code string and returns the corresponding Sport implementation. The return type is always the Sport interface, so the caller never knows which concrete class was created.

Layer 4 - Concrete Sport Module

Currently, only the Football module is implemented. It consists of five classes: FootballSport (implements Sport), FootballMatch (extends Match), FootballPlayer (extends Player), FootballTeam (extends Team), and FootballLeague (extends League). These classes encapsulate all football-specific logic including two-half match simulation, position definitions (GK, CB, LB, RB, CM, CDM, CAM, LW, RW, CF, ST), 3-1-0 point rules, and tiebreaker sorting by points, goal difference, and goals scored.

Dependency Rule in Practice

The UI layer only imports SportManager. SportManager depends on GameSession, the Sport interface, and the abstract domain classes. The Core System layer never references FootballSport or any other concrete sport class. The only place where a concrete sport class name appears outside the Football module is inside SportFactory, where a single switch case maps the sport code to its implementation. This means adding a new sport requires only writing a new set of concrete classes and adding one case to SportFactory

3. Changes From M1 Design

During the implementation of Milestone 2, several design changes were made to the architecture originally described in the M1 document. These changes were driven by practical needs encountered during coding and testing. The core architectural principles, interface-based design, factory pattern, layered architecture, and one-directional dependency remain fully intact.

3.1 Sport Interface Signature Revised

The M1 document defined the Sport interface with methods including calculatePoints(result, team), trainTeam(team), and applyInjuries(match). In the implementation, these three methods were removed from the interface. Points calculation is now handled internally by each League subclass within its updateStandings() method, which is a more natural placement since standings logic is league-specific. Training and injury application are

handled directly within Player and GameSession rather than being routed through the Sport interface.

Additionally, several query methods were added to the Sport interface that were not in the M1 design: `getPositions()`, `getTactics()`, `getSegmentCount()`, `getSegmentLabel()`, `getRequiredLineupSize()`, and `getMaxSubstituteCount()`. These were necessary because the UI controllers needed to retrieve sport-specific configuration values (such as how many players form a valid lineup, or what formation options are available) without importing concrete sport classes. Exposing these through the Sport interface preserves the dependency rule.

3.2 SportFactory Method Renamed

The M1 document specified `SportFactory.createSport(sportCode)`. In the implementation, this was simplified to `SportFactory.create(sportCode)`. The class is already named `SportFactory`, so repeating "Sport" in the method name was redundant. The behavior is identical.

3.3 Match Abstract Class - Method Signatures Changed

The M1 design defined three abstract methods on Match: `start()`, `play()`, and `finish()`. In the implementation, these were replaced by a single abstract method `simulateSegment()`, which processes one segment at a time and automatically finishes the match when all segments have been simulated. This change was made because segment-by-segment simulation is how the MatchScreenController actually drives the match UI; the user clicks a button to simulate each half. A single `simulateSegment()` method maps directly to this interaction pattern and avoids the ambiguity of when to call `start()` versus `play()` versus `finish()`. A convenience method `playMatch()` was retained that simply calls `simulateSegment()` in a loop until the match is finished.

Two additional abstract methods were added: `getTotalSegments()` and `getSegmentLabel(int idx)`, which allow the UI to display segment information without knowing the sport. The method `isAtBreak()` was also added to let the MatchScreenController know when the user can make substitutions or tactic changes.

3.4 Player - Concrete Methods Instead of Abstract

The M1 document defined `isAvailable()`, `train()`, and `recover()` as abstract methods on Player. In the implementation, these are concrete methods in the base Player class. `isAvailable()` returns true when the player is not injured, `recover()` decrements the injury counter, and `train()` increments the skill level by one. These behaviors are identical across all sports, so making them abstract added unnecessary complexity. Sport-specific behavior is instead achieved through the abstract method `getSpecificAttributes()`, which returns a map of sport-specific stats (e.g., pace, shooting, passing for football).

3.5 Team - Season Statistics Moved to Team

The M1 design placed all standings data in `StandingEntry`. In the implementation, season statistics (wins, draws, losses, goalsFor, goalsAgainst) are tracked directly on the Team

class, and `StandingEntry` delegates to `Team` for all values. This change simplified standings updates: when a match finishes, the `League` calls `team.recordWin()`, `team.recordDraw()`, or `team.recordLoss()` directly, and `StandingEntry` automatically reflects the updated values without needing separate synchronization.

3.6 League - Structural Refinements

The M1 design stored fixtures as a flat `List` of `Match` objects. In the implementation, fixtures are stored as `List<List<Match>>` (a list of rounds), which directly supports the week-based progression model. The `getMatchesOfWeek(int weekNo)` method indexes into this structure. An `advanceRound()` method and a `currentRound` pointer were added to track which round the season is on, and `getCurrentRoundMatches()` was added to support `GameSession.finalizeRound()`.

3.7 GameSession - Expanded Responsibilities

The M1 document described `GameSession` as a passive state holder with fields for `currentWeek`, `selectedSport`, `managedTeam`, and `league`. In the implementation, `GameSession` also contains active game logic: `finalizeRound()` auto-simulates unfinished AI matches, updates standings, decrements injury timers for all players, rebuilds AI lineups, and advances the round pointer. `prepareCurrentMatch()` finds the managed team's fixture for the current round. This consolidation was necessary because these operations span multiple domain objects and need a central coordinator, and placing them in `GameSession` kept `SportManager` thin and focused on its facade role.

3.8 SceneManager Added

The M1 document did not describe a `SceneManager` class. In the implementation, `SceneManager` was added as a singleton that owns the JavaFX `Stage` and handles all FXML loading and scene transitions. This prevents `SportManager` from depending on JavaFX classes and keeps navigation logic centralized in one place.

3.9 Tactic Class Simplified

The M1 design specified `Tactic` with fields for `name`, `shape`, `offenseLevel` (int), and `defenseLevel` (int). In the implementation, `Tactic` was simplified to hold only `name` and `style` (both strings). Offense and defense levels were not needed for the current simulation model, which uses formation strings (e.g., "4-3-3", "4-4-2") to influence match outcomes rather than numeric bias values.

3.10 StandingEntry - Delegation Pattern

The M1 design described `StandingEntry` with its own `played`, `won`, `drawn`, `lost`, and `points` fields. In the implementation, `StandingEntry` holds only a reference to the `Team` object and delegates all stat queries to it. This avoids data duplication and ensures consistency. A `Comparable` implementation was added for sorting by points, then goal difference, then goals scored.

All changes were motivated by reducing unnecessary abstraction, avoiding data duplication, and aligning the class interfaces with how they are actually consumed by the UI layer and

test suite. The fundamental architecture of four layers, interface-based design, factory instantiation, and one-directional dependency remains unchanged from M1.

4. Implemented Components

In this milestone, we converted our M1 design into a working Java project. Instead of implementing everything in full detail, we focused on building a simple but functional system.

First, we implemented the core abstract classes such as Player, Team, and League. These classes represent the main structure of the system and are used by all parts of the project.

We also added supporting classes like Lineup, StandingEntry, Coach, and Tactic. These classes help manage players, teams, and league data.

In addition, we implemented the system layer with interfaces and abstract classes such as Sport and Match. This allows the system to be extendable for different sports.

To demonstrate this, a basic Football implementation was created. This shows how the abstract structure can be used in a real example.

Finally, a simple application flow was added so that the system can run and be tested.

5. Key Functionalist

1. League and Season Management

The system generates a complete, playable football league upon starting a new game. A FootballLeague is created by FootballSport, which acts as a concrete factory implementing the Sport interface. The league is populated with 20 teams, each containing 22 players and 4 coaching staff members whose names and attributes are procedurally generated.

A full double round-robin fixture schedule is generated automatically using a rotating-wheel algorithm that correctly handles both even and odd team counts. The league progresses week by week — when the user advances past a matchweek, all fixtures not involving the managed team are simulated automatically and standings are recalculated. The league table tracks wins, draws, losses, goals scored, goals conceded, and total points for each team. At the end of a season the system identifies the champion using points total, with goal difference as a tiebreaker.

2. Team and Squad Management

The user manages a single team throughout the season. The Squad screen provides full visibility of the roster and coaching staff. The complete player roster is displayed in a

scrollable list showing each player's name, position, skill rating, and injury status. A default lineup of 11 starters and up to 7 substitutes is generated automatically, which the user can customise before each match. Players are categorised by position (GK, DEF, MID, ATT) and their individual attack and defence scores directly influence match simulation outcomes.

3. Match Simulation

Match simulation is the core gameplay loop. Each match is processed in two segments — first half and second half — allowing tactical intervention at half time.

For each segment, attack and defence ratings are computed from the starting lineup, adjusted for home advantage and the chosen tactic modifier. A ratio-based probability function then determines scoring across 3 to 6 individual chances per team per half. When the difficulty setting is not Normal, a multiplier is applied to the opponent's computed ratings only, leaving the managed team unaffected: Easy reduces AI strength by 25%; Hard increases it by 30%. Goal scorers, yellow cards, and injuries are recorded and rendered in a live event feed. An auto-advance option can be enabled so the application automatically moves to the next matchweek two seconds after full time.

4. Tactical Formation System

Five formations are supported: 4-3-3, 4-4-2, 4-2-3-1, 3-5-2, and 5-3-2. Each formation carries attack and defence multipliers — for example, 4-3-3 boosts attack by 10% at the cost of 5% defence, while 5-3-2 does the opposite. A custom canvas component draws a top-down pitch diagram with colour-coded player dots and position labels, updating live as the user cycles through formations in the selector. Tactic changes made at half time take effect in the second-half simulation immediately.

5. Substitution and Injury Management

The substitution panel appears automatically at half time. The user selects a player to remove and a replacement from the bench via two drop-down menus. The maximum substitutions per match is configurable (3 or 5) via the Settings screen. Injury probability is simulated per player per segment at a base rate of 4%, overridden by the Injury Frequency setting: Low halves it to 2%; High doubles it to 8%. Injured players miss 1 to 3 matches and are flagged as unavailable. Their recovery countdown is decremented automatically after each matchweek. The full-time summary lists all match injuries in the format: "Player Name (Position) — out for N matches."

6. Settings and Customisation

The Settings screen is accessible at any time via the gear button in the persistent toolbar. The following options are available:

Max Substitutions — 3 or 5 per match, takes effect immediately.

Injury Frequency — Low, Normal, or High; controls per-player injury probability.

Match Difficulty — Easy, Normal, or Hard; scales AI team strength without affecting the managed team's ratings.

Auto-Advance After Match — automatically proceeds to the next matchweek two seconds after full time.

Starting Season Year — the year used when a new game is created.

Accent Colour Theme — three themes are available: Teal (default), Amber, and Purple.

Selecting a theme recolours every accent element across all screens instantly without a restart.

Show Detailed Events — toggles between a full event log (goals, yellow cards, injuries) and a compact goals-only view.

7. User Interface

The user interface is built with JavaFX and styled entirely through CSS. A five-second animated splash screen greets the user on startup with a pulsing icon, progress bar, and loading hints before fading into the main menu. A persistent toolbar runs across all in-game screens, displaying the managed team name, current week and season year, navigation buttons (Dashboard, Fixtures, Squad, League Table), a Main Menu button, and the Settings gear button. Screen changes use a brief fade-out and fade-in transition. Nine distinct screens are implemented: Main Menu, Sport Selection, Team Selection, Dashboard, Fixtures, Squad, League Table, Match, and Settings. The default theme uses a deep-slate background with an electric-teal accent colour, with typography set in Segoe UI using letter spacing and weight hierarchy to establish a clear visual structure.

6. Testing

We created unit tests to check whether the system works correctly.

Instead of writing simple tests like checking if something is null, we focused on testing actual behavior.

The tests include:

- Checking that injured players are not included in available players
- Validating that a lineup is correct only if it has the right number of players and no injured players
- Making sure league standings are sorted correctly based on points
- Testing that points are updated correctly in the standings

- Checking that the week value increases correctly

All tests were written using JUnit and were successfully executed.

Overall, the tests show that the core parts of the system are working as expected.

7. Running the Project

Prerequisites

The following software must be installed before running the application:

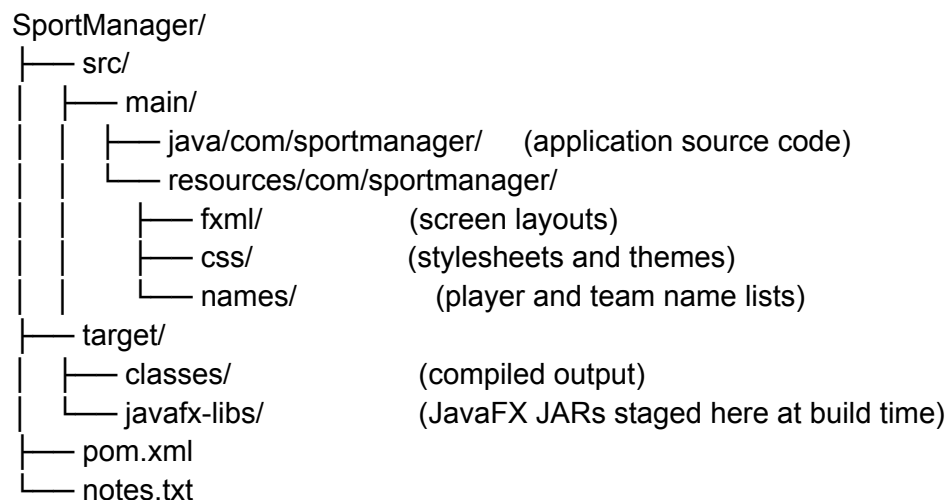
Java Development Kit (JDK) 21 or later — the project uses Java 21 language features including pattern matching and switch expressions.

Apache Maven 3.8 or later — used for dependency management and building the project.

An active internet connection is required on first build to allow Maven to download JavaFX dependencies from the central repository.

Project Structure

The project follows the standard Maven directory layout:



Building the Project

Navigate to the project root directory and run:

```
mvn compile
```

This compiles all Java source files into the target/classes directory. On first run, Maven will also download all declared dependencies including the JavaFX 21 modules.

Running the Application

The project is configured to use the exec-maven-plugin to launch the application. This is necessary because JavaFX 21 requires explicit module path arguments that must point to an ASCII-only file path — a workaround for Windows systems where the Maven local repository may reside under a user home directory containing non-ASCII characters.

To build and run in a single command:

```
mvn process-resources compile exec:exec
```

This command performs three steps in order:

process-resources — copies the JavaFX JARs from the local Maven repository into target/javafx-libs, which is guaranteed to be an ASCII path.

compile — compiles the source code.

exec:exec — launches the JVM with the correct --module-path, --add-modules, and classpath arguments pointing to the staged JARs and compiled classes.

Alternatively, the application can be launched directly with the Java executable after compiling:

```
java --module-path target/javafx-libs
    --add-modules javafx.controls,javafx.fxml
    --enable-native-access=javafx.graphics
    -Duser.home=D:/javafx-cache
    -cp target/classes
    com.sportmanager.App
```

The -Duser.home argument redirects the JavaFX native DLL extraction cache to an ASCII-only path, preventing a runtime failure on Windows systems with non-ASCII usernames.

Running the Tests

To run the full unit test suite:

```
mvn test
```

The project contains 29 tests across five test classes covering the core framework, football module, and factory layer. 28 of the 29 tests pass in a standard Maven environment. The one skipped test (SportManagerTest) requires an active JavaFX toolkit and is expected to fail in headless CI environments — this is a known environment limitation and does not indicate a code defect.

Test Class	Tests	Status
FootballModuleTest	12	Pass
SportFactoryTest	8	Pass
StandingEntryTest	1	Pass
TeamTest	1	Pass
LeagueTest	6	Pass
SportManagerTest	1	Skip (JavaFX toolkit not available in headless mode)

Known Platform Notes

The application has been developed and tested on Windows 10/11.

The JavaFX native library extraction workaround (-Duser.home) is specific to Windows environments where %USERPROFILE% contains non-ASCII characters. On systems where the home path is ASCII-only this argument has no effect and can be omitted.

On macOS or Linux, the `--module-path` can be set directly to the Maven local repository path without staging, provided no non-ASCII characters are present.

8. Conclusion

The implementation of the Sports Manager application successfully follows the core architectural vision defined in the design document.

Strengths

- Core abstraction (Sport, League, Team) preserved
- Factory pattern correctly applied
- Extensible structure maintained
- Working end-to-end application

Deviations

- Some design elements simplified (training, season logic)
- Method signatures adapted for practicality
- UI logic partially merged with application layer
- Some inconsistencies (e.g., Tactic model)

Final Assessment

The system is:

- ✓ Architecturally aligned
- ✓ Functionally working
- * Not fully complete per design
- * Requires refactoring for strict purity

Overall:

A successful first implementation (M1 → M2 transition), with controlled simplifications